Predicting Wine Quality Using Machine Learning

DAT158 – Assignment 2. Name(s**): Erik Kristian Finsås (669806)**. Date: 30/10/2026

1: DESCRIBE THE PROBLEM

Scope

The aim of this project is to see whether we can predict a wine's quality score from its chemical properties. I chose the Wine Quality dataset because it contains measurements for both red and white wine, as well as a quality score for each sample. Our task is classification: instead of predicting any decimal number, the model chooses one of the quality scores that appears in the training data.

Wine quality is normally judged by people who taste the wine and consider things like smell, flavour and balance. Producers also use laboratory measurements to check whether a wine meets their standards. Both approaches are useful, but they do different things. A chemical test gives measurable values, while a tasting gives a human judgement. Our idea is to investigate how much of that judgement a machine learning model can estimate from the chemical data alone.

Without machine learning, someone could compare the measurements with past samples, use simple rules, or ask an expert to taste each wine. These methods may work, but they can be slow when there are many samples. A trained model could produce a quick estimate after the laboratory values have been entered. We do not expect it to replace professional tasting. It is more realistic to see it as an extra tool that helps someone decide which wines deserve closer inspection.

We work with Python, pandas and scikit-learn, and we use Gradio to make a simple interface. The intended user is someone who already has the required chemical measurements, for example a student testing the model or a producer exploring a batch of wine. This is a course project rather than a finished commercial system. Before real-world use, the model would need testing on new wines from other producers and a way to check whether its predictions remain reliable.

For the project itself, I mainly need a computer that can run a Jupyter notebook and access the two CSV files. No special hardware is required. If the system were developed further, I would also need reliable measurement procedures and feedback from people who understand wine assessment.

Metrics

Our main evaluation metric is accuracy: the share of wines in the test set for which the predicted quality score is correct. For example, a prediction of 5 when the actual score is 6 counts as incorrect, even though it is only one point away. This is one limitation of using accuracy for quality scores.

We also use precision, recall and F1-score from the classification report. These tell us more about how the model performs for individual scores. A confusion matrix helps us see whether the model tends to mix up neighbouring scores. This matters because the dataset contains many more wines with common scores than with very low or very high scores.

To decide whether the model is learning anything useful, we compare it with a Dummy Classifier that always predicts the most frequent quality score. Our minimum goal for the assignment is to beat this baseline by a noticeable margin on the held-out test data. In a real application, we would also care about the time saved, the frequency of large prediction errors and whether users find the estimates useful. We have not measured these business outcomes in this project.

2: DATA

I used the Wine Quality dataset from the UCI Machine Learning Repository. The red wine file contains 1,599 rows and the white wine file contains 4,898 rows, for 6,497 samples in total. Each row represents one wine sample. The original files contain 11 chemical measurements and one target column called quality. The measurements include fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free and total sulfur dioxide, density, pH, sulphates and alcohol.

The quality column is the label we want the model to predict. These labels were already included in the dataset, so we did not collect or assign them ourselves. They are based on sensory evaluations, which means they may contain some subjectivity. I use the labels as provided, but this is worth remembering when interpreting the model's performance.

I first loaded the two CSV files with pandas, using a semicolon as the separator. We inspect the first rows, dataset shapes and descriptive statistics. We also check for missing values; the notebook outputs show none in either file. We then add a wine_type column to distinguish red from white wine and combine the datasets into one table. For training, red is encoded as 0 and white as 1.

Before modelling, we look at the distribution of quality scores, compare alcohol content between red and white wine, and create a correlation heatmap. These plots help us understand the data rather than simply feeding it straight into a model. The quality scores are not evenly distributed, which is important when we later interpret the classification results.

Our input features are the chemical measurements plus the encoded wine type. The target is quality. For the main experiment, we use an 80/20 train-test split with random_state=3 and stratification by quality. Stratification helps preserve approximately the same score distribution in both sets. We do not scale the variables because our main model is a Random Forest, which does not require standardised input values in the same way some other models do.

There is little direct privacy risk because the data contains measurements of wine rather than personal information. A bigger concern is whether the dataset represents wines outside this collection. We should not assume the model will work equally well for all regions, producers or future batches. We also need to avoid presenting a predicted score as an objective guarantee of quality.

3: MODELING

I chose a RandomForestClassifier as our main model. A Random Forest trains many decision trees and combines their predictions. This seems suitable for our dataset because relationships between chemical properties and quality might not be simple or linear. We first train the model on the combined dataset and evaluate it on the test set.

For comparison, we train a DummyClassifier using the most_frequent strategy. This model ignores the chemical measurements and always predicts the most common score. It gives us a useful starting point: if the Random Forest cannot beat this baseline, there would be little reason to use the more complicated model.

The results for the main split:

Model	Accuracy
Dummy Classifier (most frequent)	43.62%
Random Forest (initial)	69.15%
Random Forest (300 trees, max depth 20)	68.92%

The initial Random Forest is around 25.5 percentage points above the baseline. This suggests that the chemical measurements contain information that helps predict the given quality labels. However, an accuracy of about 69% also means that roughly three out of ten test predictions do not match the exact score. The model is therefore useful as an experiment, but it is not reliable enough to treat as a final judgement of wine quality.

I also tried a second Random Forest with 300 trees and a maximum depth of 20. Its recorded accuracy is 68.92%, slightly lower than the first model. Increasing the number of trees or changing the depth does not automatically improve performance. We would need more systematic testing, ideally with cross-validation on the training data, before deciding which settings are best.

The classification report shows that the model works better for common quality scores, particularly 5 and 6, than for rare scores such as 3 and 9. This makes sense because the model has far fewer examples of the rare classes to learn from. The confusion matrix helps illustrate where the incorrect predictions occur. We also create a feature-importance chart to investigate which inputs the forest uses most. Feature importance is useful for interpreting the model, but it does not prove that any particular chemical property causes a wine to receive a higher score.

We then make an additional comparison by training separate models for red wine, white wine and the combined dataset. In that experiment, wine_type is removed from the inputs and a different random split is used. The notebook reports 65.94% accuracy for red wine, 68.98% for white wine and 67.00% for the combined data. These results are interesting, but they should not be directly compared with the main 69.15% result because the splitting procedure is different. They also do not establish that one type of wine is inherently easier to predict. (Breiman, 2001)

4: DEPLOYMENT

To demonstrate how the model could be used, I created a simple Gradio application in the notebook. It shows input fields for the chemical measurements and a dropdown for wine type. When the user enters values, the application creates a row with the same feature columns used for training and asks the Random Forest to predict a quality score. The result is displayed as text.

The interface makes the project easier to demonstrate because someone can try the model without writing Python code. However, it also has limitations. The user must know the chemical measurements, so it is not an app that can judge a wine from its name or a photograph. We would also need to check that the entered values are reasonable and that the order and meaning of the inputs stay consistent with training.

Maintenance would involve checking whether the model still performs well on newer wine samples. If the types of wines or laboratory measurements changed, the model might become less accurate. We would also want to collect feedback about incorrect predictions and retrain the model when enough suitable new data becomes available.

References

Breiman, L. (2001). Random Forests. *Machine Learning*, 5–32.

Chawla, N. V. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 321–357.

developers, S.-l. (2026, October 30). *classification_report*. Retrieved from scikit-learn: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

Gradio. (2026, October 30). *Quickstart*. Retrieved from Gradio: https://gradio.app/guides/quickstart

Pedregosa. (2011). Journal of Machine Learning Research. *Scikit-learn: Machine Learning in Python*, 2825–2830.